Multimodal Insurance Creator

of a Multi-Agent Insurance System

Course Name: DevOps

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Mustafa shaikh	EN22CS301624
02	Naman Mehta	EN22CS301629
03	Naman Raghuvanshi	EN22CS301632
04	Nandini Sharma	EN22CS301635
05	Nishkarsh Sharma	EN22CS301659

Group Name: DO-20

Project Number: 09D6

Industry Mentor Name: Rishab Sir

University Mentor Name: Prof. Ajaz khan

Academic Year: 2025-2026

1. Introduction

The Multimodal GenAI Application for Insurance Concept Visualization is designed to provide an intuitive and interactive way to understand complex insurance concepts. The system leverages Generative AI by integrating Large Language Models (LLMs) and image generation models to produce both textual explanations and visual representations from a single user input.

The high-level design focuses on the overall architecture, system components, and interactions between different layers, ensuring scalability, flexibility, and efficient processing of multimodal outputs.

1.1 Scope of the Document

This document presents the High-Level Design (HLD) of the system, describing the overall architecture and major system components without going into implementation-level details.

The scope includes:
• High-level system architecture and layers
• Overview of major components and their roles
• Interaction between frontend, backend, and AI services
• End-to-end workflow of multimodal content generation
• Deployment and integration overview

1.2 Intended Audience

• **System Architects** – To understand overall system structure
• **Developers** – To get a high-level understanding before implementation
• **AI/ML Engineers** – To understand model integration flow
• **DevOps Engineers** – To understand deployment architecture
• **Project Evaluators** – To assess system design and approach

1.3 System Overview

The system enables users to input prompts related to insurance concepts such as risk, coverage, or policy benefits. The input is processed by a backend service that interacts with two main AI components: an LLM for generating textual explanations and an image generation model for producing visual outputs.

Both outputs are combined and presented through a unified interface. The system follows a layered architecture, ensuring separation between user interaction, processing logic, and AI services. This design enables scalability, maintainability, and efficient handling of multimodal data.

2. System Design

The system is designed using a high-level, layered architecture that integrates user interaction, application processing, and AI-driven services to generate multimodal outputs. It combines the capabilities of text generation and image generation models to provide a comprehensive visualization of insurance concepts.

At a high level, the system consists of three main layers: the presentation layer, the application layer, and the AI service layer. The presentation layer handles user interaction through a web-based interface, allowing users to input prompts and view results. The application layer acts as the core processing unit, managing request handling, orchestration, and communication between components. The AI service layer includes external models responsible for generating textual and visual outputs.

The system follows an API-driven approach, where the backend communicates with external AI services to process user inputs. Both text and image generation processes are executed in parallel to improve performance and reduce response time. The outputs are then combined and delivered to the user in a unified format.

Key Aspects of System Design:

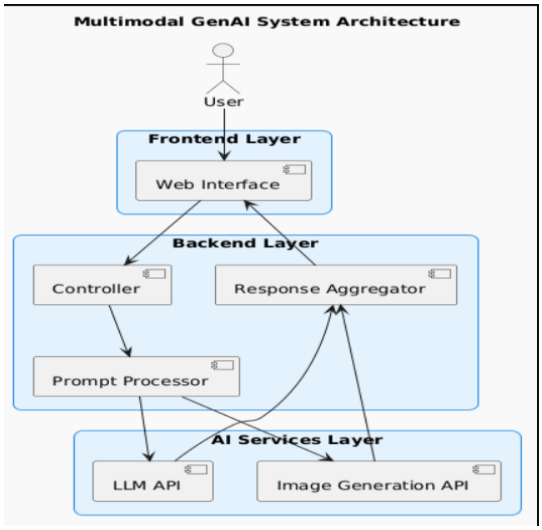
• **Layered Architecture** – Separation of frontend, backend, and AI services for better organization

• **API Integration** – Communication with external LLM and image generation services through APIs

• **Parallel Processing** – Simultaneous execution of text and image generation tasks

• **Scalability** – Ability to integrate additional models or handle increased load

• **Modularity** – Independent components for easier maintenance and upgrades



2.1 Application Design

The application is designed using a layered and modular approach to ensure clear separation of responsibilities and efficient processing of multimodal outputs. At a high level, the system is divided into the frontend, backend, and AI service components, each performing a specific role in the overall workflow.

The **frontend layer** provides a user-friendly interface where users can input prompts related to insurance concepts and view the generated results. It is responsible for capturing user input and displaying both textual and visual outputs in a structured format.

The **backend layer** acts as the central controller of the system. It handles incoming requests, processes user input, and coordinates communication with external AI services. The backend ensures that the input is properly formatted and manages the execution of multiple tasks in parallel.

The **AI service layer** consists of external models accessed via APIs. One service is responsible for generating textual explanations using a Large Language Model, while another service generates visual representations such as infographics using an image generation model.

The backend aggregates the outputs from both AI services and sends a unified response to the frontend for display. This design ensures smooth interaction between components and efficient generation of multimodal content.

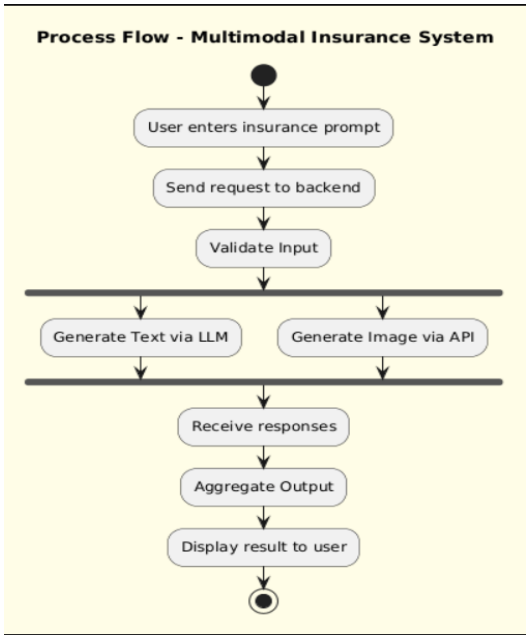
Key Elements:

• **User Interface** – Handles input and displays results
• **Backend Controller** – Manages request processing and orchestration
• **LLM Service** – Generates textual content
• **Image Generation Service** – Produces visual outputs
• **Response Aggregation** – Combines text and image outputs

1.2. Process Flow

The process flow describes the high-level sequence of steps through which the system generates multimodal outputs from a user-provided prompt. The system is designed to handle requests efficiently by executing text and image generation processes in parallel and combining the results into a unified response.

At a high level, the workflow begins with user interaction and progresses through backend processing, AI service integration, and final output delivery.



Process Steps:

• The user enters an insurance-related prompt through the web interface

• The frontend sends the request to the backend service

• The backend validates and processes the input

• The processed input is sent simultaneously to:
– LLM service for text generation
– Image generation service for visual output

• Both services generate their respective outputs in parallel

• The backend receives responses from both services

• The outputs are aggregated into a structured format

• The final combined response is sent back to the frontend

• The frontend displays the textual explanation along with the generated visual

1.3 Information Flow

The information flow describes how data moves across different layers of the system, from user input to final output. The system ensures smooth and structured communication between the frontend, backend, and AI service layers using API-based interactions.

At a high level, the user input is captured by the frontend and transmitted to the backend. The backend processes the request and communicates with external AI services to generate text and image outputs. These outputs are then aggregated and returned to the user.

Information Flow Steps:

• User → Frontend (Input submission)
• Frontend → Backend (Request transmission)
• Backend → AI Services (API calls to LLM and Image model)
• AI Services → Backend (Generated outputs)
• Backend → Frontend (Aggregated response)
• Frontend → User (Final display)

1.4 Components Design

The system is composed of high-level components that collaborate to deliver multimodal outputs. Each component performs a specific role within the system architecture.

Components:

• **User Interface** – Captures user input and displays results
• **Backend Controller** – Manages request processing and system workflow
• **LLM Service** – Generates textual descriptions
• **Image Generation Service** – Produces visual representations
• **Response Aggregator** – Combines outputs into a unified response

The design ensures loose coupling between components, allowing independent updates and scalability.

1.5 Key Design Considerations

• **Modularity** – System components are organized into separate configurations for easier maintenance.
• **Scalability** – The container orchestration platform allows the application to scale according to workload.
• **Reliability** – Deployment configurations ensure consistent application execution across environments.

1.6 API Catalogue

The system relies on APIs to enable communication between the backend and external AI services. These APIs provide the functionality required to generate textual and visual outputs.

APIs Used:

• **LLM API** – Generates detailed textual explanations from user prompts

• **Image Generation API** – Produces visual representations such as infographics based on input text

• **Backend API** – Handles communication between frontend and backend services

Key Characteristics:

• REST-based communication
• JSON request and response format
• Secure API access with authentication
• Scalable and easily replaceable services

1.7 Use Case Overview

The Use Case Overview describes how users and system components interact to achieve the functionality of the application. It highlights the primary actions performed by users and the system at a high level.

Actors:

• **User** – Interacts with the system to generate and view insurance concept visualizations
• **Admin / Developer** – Manages system updates and deployment (optional role in HLD)

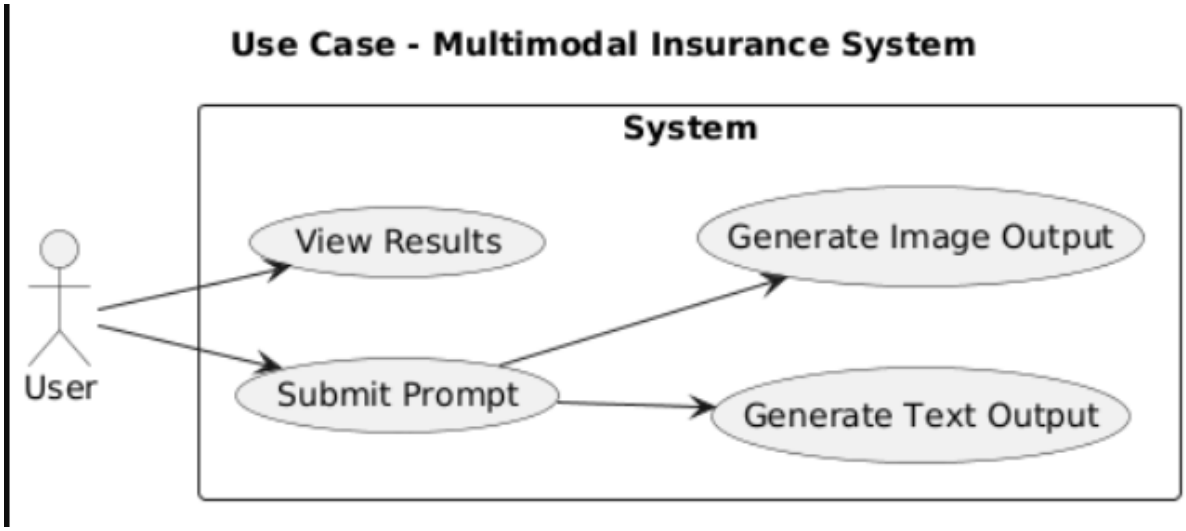
Use Cases:

User Use Cases:

• Submit prompt for insurance concept
• View generated textual explanation
• View generated visual representation
• Interact with the system for multiple queries

System Use Cases:

• Process user input
• Generate text using LLM
• Generate image using AI model
• Aggregate and present output



Use Case Flow Summary:

The user submits a prompt, which is processed by the backend. The system generates both textual and visual outputs using AI services, combines them, and presents the final result to the user.

2. Data Design

The data design defines how the system manages textual prompts, generated outputs, and configuration data used during execution. The system handles both input prompts and multimodal outputs (text + images), along with configuration files required for deployment and integration with AI services.

2.1 Data Model

The data model defines the structure of the information used within the system components.

Main Data Elements include:

• **User Prompt Data** – Contains the input text provided by the user describing an insurance concept (e.g., risk, coverage)

• **Generated Text Output** – Stores the descriptive narrative generated by the LLM based on the user prompt

• **Generated Image Output** – Stores the visual representation (infographics or diagrams) generated by the image model

• **API Configuration Data** – Includes API keys, endpoints, and parameters required to interact with AI services such as OpenAI API

• **Application Source Code** – Contains backend logic, frontend files, and integration modules

• **Container Configuration** – Defines how the application is packaged and deployed in a containerized environment

2.2 Data Migration

Data migration in this system involves updating prompt templates, AI configurations, and generated content workflows.

• New prompt formats or templates can be added to improve output quality

• API configurations can be updated when switching or upgrading AI models

• Generated outputs can be modified or extended without affecting system flow

• The modular design allows integration of new data sources or AI services without major changes

3. Interfaces

• **User Interface** – Web-based interface for entering prompts and viewing generated text and images

• **AI Service Interface** – API-based interaction with LLM and image generation services such as OpenAI API

4. State and Session Management

State management in the system is minimal and request-based.

• Each user request is processed independently (stateless design)

• Input prompts and outputs are handled per session without persistent storage

• Temporary state is maintained during API calls and response aggregation

• Stateless architecture improves scalability and simplifies deployment

5. Caching

Caching is used to improve performance and reduce repeated processing.

• **Response Caching** – Frequently used prompts can return cached results

• **API Response Caching** – Reduces repeated calls to AI services

• **Container Caching** – Docker layer caching improves build performance

6. Functional Requirements

The functional requirements define the capabilities of the multimodal GenAI system.

6.1 Prompt Processing

• The system must accept user input related to insurance concepts

• The input must be validated and formatted before processing

• The same prompt must be used for both text and image generation

6.2 Text Generation

• The system must generate descriptive explanations using an LLM

• The output should be structured and relevant to the input concept

• The system must interact with AI services via APIs

6.3 Image Generation

• The system must generate visual representations of the concept

• The output may include infographics or conceptual diagrams

• The system must use image generation APIs for processing

6.4 Multimodal Output Integration

• The system must combine text and image outputs into a single response

• The output should be presented in a structured and user-friendly format

• Both outputs must correspond to the same input prompt

6.5 Deployment and Execution

• The application must be containerized using Docker

• The system should support automated deployment workflows

• The application must be accessible via a web interface

7. Non-Functional Requirements

The functional requirements define the capabilities of the multimodal GenAI system.

6.1 Prompt Processing

• The system must accept user input related to insurance concepts

• The input must be validated and formatted before processing

• The same prompt must be used for both text and image generation

6.2 Text Generation

• The system must generate descriptive explanations using an LLM

• The output should be structured and relevant to the input concept

• The system must interact with AI services via APIs

6.3 Image Generation

• The system must generate visual representations of the concept

• The output may include infographics or conceptual diagrams

• The system must use image generation APIs for processing

6.4 Multimodal Output Integration

• The system must combine text and image outputs into a single response

• The output should be presented in a structured and user-friendly format

• Both outputs must correspond to the same input prompt

6.5 Deployment and Execution

• The application must be containerized using Docker

• The system should support automated deployment workflows

• The application must be accessible via a web interface

8. References

Problem Statement: **multimodel insurance creator**

Project Group: **D6 - Group 09**.

Flask Documentation
https://flask.palletsprojects.com/

• OpenAI API –
OpenAI API Documentation

• Docker –
Docker Documentation

• GitHub –
GitHub Documentation

• Stable Diffusion –
Stable Diffusion Overview